GitHub Repo Link:
https://github.com/Aadeleye11/Project-2_Assignment_Formative_C

Project 1: Dynamic Academic Performance and Risk Analysis Engine

Complete C Source Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <limits.h>
#include <ctype.h>

#define STUDENT_ID "003"
#define PASS 60.0

typedef struct {
   int id;
   char name[60], course[60];
   double grade;
} Student;

typedef struct {
   Student *list;
   size_t count;
} Database;

/* Read bounded strings and reject excess characters. */
int input(const char *message, char *s, size_t size) {
   printf("%s", message);
   if (!fgets(s, (int)size, stdin)) return 0;
   size_t n = strlen(s);
   if (n && s[n - 1] == '\n') s[n - 1] = '\0';
   else if (!feof(stdin)) {
       int c;
       while ((c = getchar()) != '\n' && c != EOF) {}
       puts("Input too long.");
```

```
        return 0;
    }
    return 1;
}
int number(const char *message, int *value) {
    char s[80], *end;
    if (!input(message, s, sizeof s)) return 0;
    errno = 0;
    long n = strtol(s, &end, 10);
    while (isspace((unsigned char)*end)) end++;
    if (errno || s == end || *end || n < INT_MIN || n > INT_MAX) {
        puts("Invalid whole number."); return 0;
    }
    *value = (int)n;
    return 1;
}
int position(Database *db, int id) {
    for (size_t i = 0; i < db->count; i++)
        if (db->list[i].id == id) return (int)i;
    return -1;
}
void heading(void) { puts("ID     Name                    Course
Grade"); }
void display(Database *db) {
    if (!db->count) { puts("No students."); return; }
    heading();
    for (Student *p = db->list; p < db->list + db->count; p++)
        printf("%-6d %-23.23s %-22.22s %5.1f\n", p->id, p->name, p->course,
p->grade);
}
void add(Database *db) {
    Student s = {0}; char grade[80], *end;
    if (!number("New ID: ", &s.id)) return;
    if (s.id <= 0 || position(db, s.id) != -1) { puts("ID must be positive and
unique."); return; }
    if (!input("Full name: ", s.name, sizeof s.name) || !s.name[0]) return;
    if (!input("Course: ", s.course, sizeof s.course) || !s.course[0]) return;
    if (!input("Grade (0-100): ", grade, sizeof grade)) return;
    errno = 0;
    s.grade = strtod(grade, &end);
    while (isspace((unsigned char)*end)) end++;
    if (errno || grade == end || *end || !(s.grade >= 0 && s.grade <= 100)) {
```

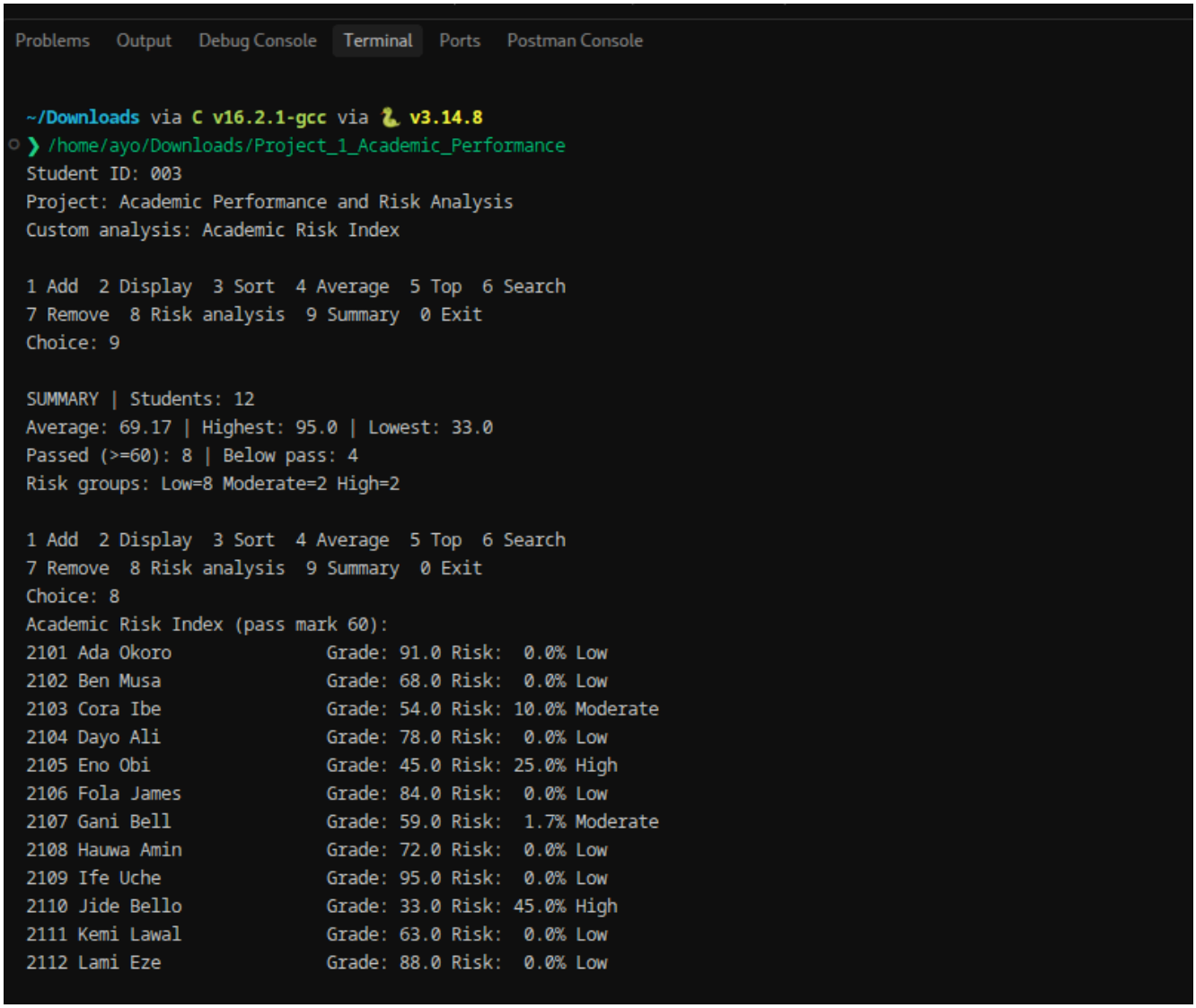
```
        puts("Invalid grade. Enter 0 to 100."); return;
    }
    Student *temp = realloc(db->list, (db->count + 1) * sizeof *temp);
    if (!temp) { puts("Memory allocation failed."); return; }
    db->list = temp;
    db->list[db->count++] = s;
    printf("Student added. Total: %zu\n", db->count);
}
int grade_order(const void *a, const void *b) {
    double x = ((const Student *)a)->grade, y = ((const Student *)b)->grade;
    return (x < y) - (x > y); /* highest first */
}
int name_order(const void *a, const void *b) {
    return strcmp(((const Student *)a)->name, ((const Student *)b)->name);
}
int id_order(const void *a, const void *b) {
    int x = ((const Student *)a)->id, y = ((const Student *)b)->id;
    return (x > y) - (x < y);
}
void sort(Database *db) {
    int option;
    if (!db->count) { puts("No students."); return; }
    if (!number("Sort: 1=grade, 2=name, 3=ID: ", &option)) return;
    int (*compare)(const void *, const void *) = NULL;
    if (option == 1) compare = grade_order;
    if (option == 2) compare = name_order;
    if (option == 3) compare = id_order;
    if (!compare) { puts("Invalid choice."); return; }
    qsort(db->list, db->count, sizeof(Student), compare);
    display(db);
}
double average(Database *db) {
    if (!db->count) return 0;
    double sum = 0;
    for (Student *p = db->list; p < db->list + db->count; p++) sum += p->grade;
    return sum / db->count;
}
void show_average(Database *db) {
    if (db->count) printf("Average: %.2f\n", average(db));
    else puts("No students.");
}
void top(Database *db) {
```

```
    if (!db->count) { puts("No students."); return; }
    Student *best = db->list;
    for (Student *p = db->list + 1; p < db->list + db->count; p++)
        if (p->grade > best->grade) best = p;
    printf("Top student: %s (ID %d), %.1f\n", best->name, best->id,
best->grade);
}
void search(Database *db) {
    int option;
    if (!number("Search 1=ID, 2=exact name: ", &option)) return;
    if (option == 1) {
        int id;
        if (!number("ID: ", &id)) return;
        int i = position(db, id);
        if (i >= 0) printf("Found: %d | %s | %s | %.1f\n", db->list[i].id,
                           db->list[i].name, db->list[i].course,
db->list[i].grade);
        else puts("Not found.");
    } else if (option == 2) {
        char name[60]; int found = 0;
        if (!input("Full name: ", name, sizeof name)) return;
        for (size_t i = 0; i < db->count; i++)
            if (!strcmp(db->list[i].name, name)) {
                printf("Found: %d | %s | %.1f\n", db->list[i].id, name,
db->list[i].grade);
                found = 1;
            }
        if (!found) puts("Not found.");
    } else puts("Invalid choice.");
}
void remove_student(Database *db) {
    int id;
    if (!number("ID to remove: ", &id)) return;
    int i = position(db, id);
    if (i < 0) { puts("Not found."); return; }
    for (size_t j = (size_t)i; j + 1 < db->count; j++) db->list[j] = db->list[j
+ 1];
    db->count--;
    if (!db->count) { free(db->list); db->list = NULL; }
    else {
        Student *temp = realloc(db->list, db->count * sizeof *temp);
        if (temp) db->list = temp; /* failed shrinking is harmless */
```

```
    }
    printf("Removed. Total: %zu\n", db->count);
}
/* Original analysis: risk = max(0, (60 - grade) / 60 * 100).
  Low: 0; Moderate: >0 and <25; High: >=25. */
double risk(double grade) { return grade >= PASS ? 0 : (PASS - grade) / PASS *
100; }
const char *level(double r) { return r == 0 ? "Low" : r < 25 ? "Moderate" :
"High"; }
void analyze(Database *db) {
    if (!db->count) { puts("No students."); return; }
    puts("Academic Risk Index (pass mark 60):");
    for (Student *p = db->list; p < db->list + db->count; p++)
        printf("%d %-23.23s Grade:%5.1f Risk:%5.1f%% %s\n",
               p->id, p->name, p->grade, risk(p->grade),
level(risk(p->grade)));
}
void summary(Database *db) {
    printf("\nSUMMARY | Students: %zu\n", db->count);
    if (!db->count) return;
    double low = db->list[0].grade, high = low;
    size_t passed = 0, moderate = 0, high_risk = 0;
    for (Student *p = db->list; p < db->list + db->count; p++) {
        if (p->grade < low) low = p->grade;
        if (p->grade > high) high = p->grade;
        if (p->grade >= PASS) passed++;
        double r = risk(p->grade);
        if (r >= 25) high_risk++;
        else if (r > 0) moderate++;
    }
    printf("Average: %.2f | Highest: %.1f | Lowest: %.1f\n", average(db), high,
low);
    printf("Passed (>=60): %zu | Below pass: %zu\n", passed, db->count -
passed);
    printf("Risk groups: Low=%zu Moderate=%zu High=%zu\n",
           db->count - moderate - high_risk, moderate, high_risk);
}
int main(void) {
    Student initial[] = {
        {2101,"Ada Okoro","Algorithms",91}, {2102,"Ben Musa","Databases",68},
        {2103,"Cora Ibe","Networks",54}, {2104,"Dayo Ali","Security",78},
        {2105,"Eno Obi","Algorithms",45}, {2106,"Fola James","Networks",84},
```

```
        {2107,"Gani Bell","Databases",59}, {2108,"Hauwa Amin","Security",72},
        {2109,"Ife Uche","Algorithms",95}, {2110,"Jide Bello","Networks",33},
        {2111,"Kemi Lawal","Databases",63}, {2112,"Lami Eze","Security",88}
    };
    Database db = {0};
    db.count = sizeof initial / sizeof initial[0];
    db.list = malloc(db.count * sizeof(Student));
    if (!db.list) { puts("Memory allocation failed."); return 1; }
    memcpy(db.list, initial, sizeof initial);
    void (*operations[])(Database *) = {
        add, display, sort, show_average, top, search, remove_student, analyze,
summary
    };
    printf("Student ID: %s\nProject: Academic Performance and Risk Analysis\n",
STUDENT_ID);
    puts("Custom analysis: Academic Risk Index");
    for (;;) {
        int choice;
        puts("\n1 Add  2 Display  3 Sort  4 Average  5 Top  6 Search");
        puts("7 Remove  8 Risk analysis  9 Summary  0 Exit");
        if (!number("Choice: ", &choice)) { if (feof(stdin)) break; continue; }
        if (choice == 0) break;
        if (choice >= 1 && choice <= 9) operations[choice - 1](&db);
        else puts("Invalid choice.");
    }
    free(db.list);
    return 0;
}
```

Sample Output from one test run



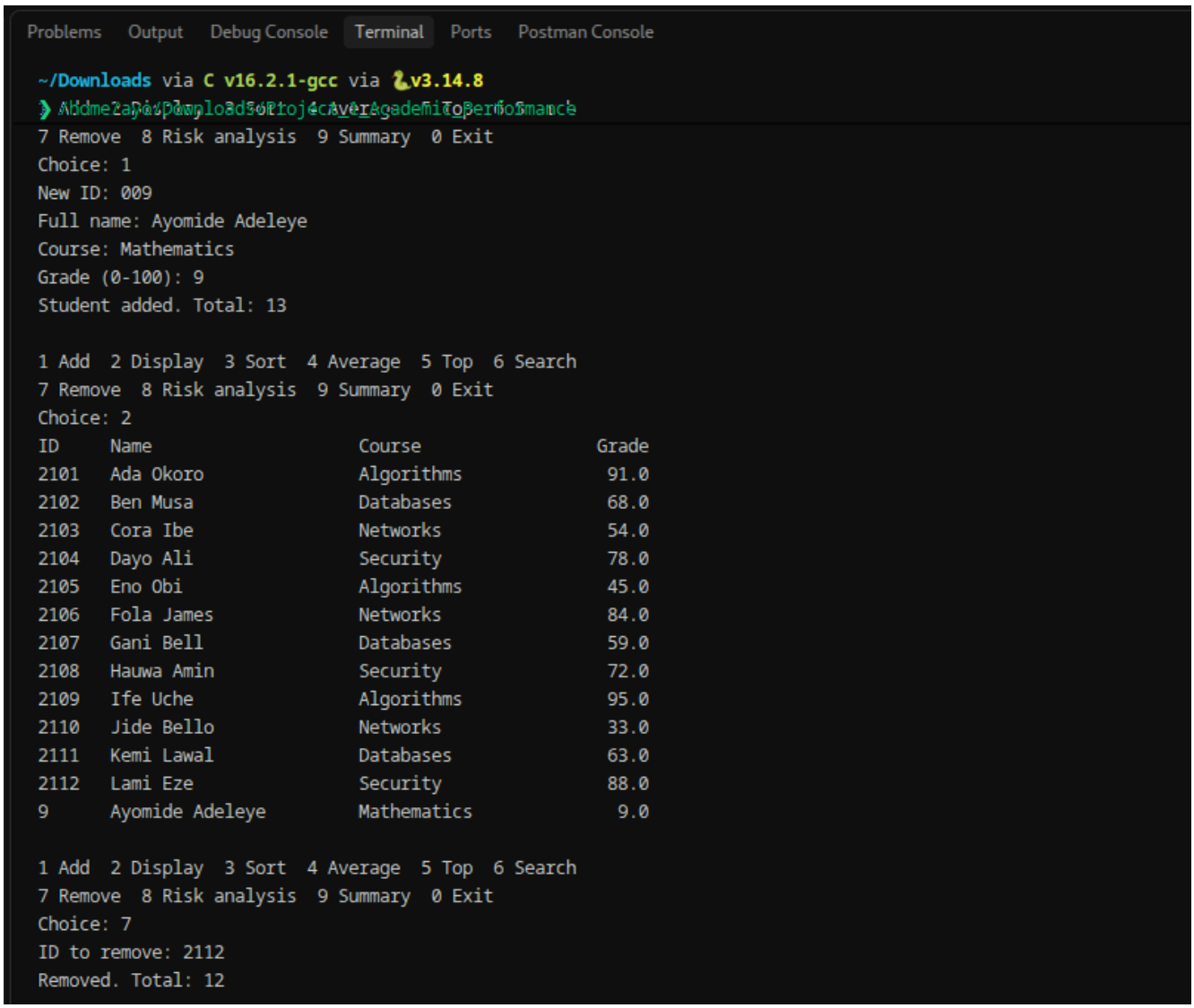
```
Problems  Output  Debug Console  Terminal  Ports  Postman Console

~/Downloads via C v16.2.1-gcc via 🐍 v3.14.8
❯ /home/ayo/Downloads/Project_1_Academic_Performance
Student ID: 003
Project: Academic Performance and Risk Analysis
Custom analysis: Academic Risk Index

1 Add  2 Display  3 Sort  4 Average  5 Top  6 Search
7 Remove  8 Risk analysis  9 Summary  0 Exit
Choice: 9

SUMMARY | Students: 12
Average: 69.17 | Highest: 95.0 | Lowest: 33.0
Passed (>=60): 8 | Below pass: 4
Risk groups: Low=8 Moderate=2 High=2

1 Add  2 Display  3 Sort  4 Average  5 Top  6 Search
7 Remove  8 Risk analysis  9 Summary  0 Exit
Choice: 8
Academic Risk Index (pass mark 60):
2101 Ada Okoro              Grade: 91.0 Risk:  0.0% Low
2102 Ben Musa               Grade: 68.0 Risk:  0.0% Low
2103 Cora Ibe               Grade: 54.0 Risk: 10.0% Moderate
2104 Dayo Ali               Grade: 78.0 Risk:  0.0% Low
2105 Eno Obi                Grade: 45.0 Risk: 25.0% High
2106 Fola James             Grade: 84.0 Risk:  0.0% Low
2107 Gani Bell              Grade: 59.0 Risk:  1.7% Moderate
2108 Hauwa Amin             Grade: 72.0 Risk:  0.0% Low
2109 Ife Uche               Grade: 95.0 Risk:  0.0% Low
2110 Jide Bello             Grade: 33.0 Risk: 45.0% High
2111 Kemi Lawal             Grade: 63.0 Risk:  0.0% Low
2112 Lami Eze               Grade: 88.0 Risk:  0.0% Low
```



Short Technical Explanation

- **Student records:** The Student structure stores an ID, full name, course and grade. The program begins with 12 example students and displays records in a formatted table.
- **Dynamic memory:** malloc() creates the initial record array, realloc() increases or reduces its size, and free() releases the memory at exit. Allocation failures are checked.
- **Pointers and sorting:** Pointer traversal is used to display and calculate records. qsort() sorts grades, names or IDs, and strcmp() compares names.
- **Function pointers:** The operations array maps menu choices to functions such as add, display, search, remove and summary.
- **Custom analysis - Academic Risk Index:** With a pass threshold of 60, risk = max(0, (60 - grade) / 60 x 100). A risk of 0 is Low, above 0 but below 25 is Moderate, and 25 or more is High. A larger score shows a greater shortfall below the passing grade.

- **Summary and validation:** The report shows the number of students, average, highest/lowest scores, pass/fail counts and risk groups. Invalid grades and duplicate IDs are rejected.

Project 2: Dynamic Text Mining and Linguistic Pattern Analyzer

Complete C Source Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>
#include <stdint.h>

#define STUDENT_ID "002"

typedef struct wordStat {
   char name[40];
   int count;
} WordStat;

typedef struct {
   char *text;
   size_t length, capacity;
   WordStat *words;
   size_t distinct, room, total, rejected;
} Analyzer;

/* L. Frank Baum, The Wonderful Wizard of Oz (1900), Chapter 1. */
const char *sample =
"Dorothy lived in the midst of the great Kansas prairies, with Uncle Henry, "
"who was a farmer, and Aunt Em, who was the farmer's wife. Their house was "
"small, for the lumber to build it had to be carried by wagon many miles. "
"There were four walls, a floor and a roof, which made one room; and this "
"room contained a rusty looking cookstove, a cupboard for the dishes, a "
"table, three or four chairs, and the beds. Uncle Henry and Aunt Em had a "
"big bed in one corner, and Dorothy a little bed in another corner. There "
"was no garret at all, and no cellar-except a small hole dug in the ground, "
"called a cyclone cellar, where the family could go in case one of those "
"great whirlwinds arose, mighty enough to crush any building in its path. "
"It was reached by a trap door in the middle of the floor, from which a "
"ladder led down into the small, dark hole.";

int input(const char *message, char *s, size_t cap) {
```

```
   printf("%s", message);
   if (!fgets(s, (int)cap, stdin)) return 0;
   size_t n = strlen(s);
   if (n && s[n - 1] == '\n') s[n - 1] = '\0';
   else if (!feof(stdin)) {
       int c; while ((c = getchar()) != '\n' && c != EOF) {}
       puts("Input too long."); return 0;
   }
   return 1;
}
int get_choice(int *choice) {
   char line[40], *end;
   if (!input("Choice: ", line, sizeof line)) return 0;
   errno = 0;
   long n = strtol(line, &end, 10);
   while (isspace((unsigned char)*end)) end++;
   if (errno || end == line || *end || n < 0 || n > 11) { puts("Invalid
choice."); return 0; }
   *choice = (int)n; return 1;
}
/* Allocate a growing text buffer instead of imposing a fixed text length. */
int append(Analyzer *a, const char *chunk) {
   size_t n = strlen(chunk);
   if (n > SIZE_MAX - a->length - 1) return 0;
   size_t needed = a->length + n + 1;
   if (needed > a->capacity) {
       size_t size = a->capacity ? a->capacity : 256;
       while (size < needed) {
           if (size > SIZE_MAX / 2) { size = needed; break; }
           size *= 2;
       }
       char *p = realloc(a->text, size);
       if (!p) { puts("Text memory allocation failed."); return 0; }
       a->text = p; a->capacity = size;
   }
   memcpy(a->text + a->length, chunk, n + 1);
   a->length += n;
   return 1;
}
int count_word(Analyzer *a, const char *word) {
   for (size_t i = 0; i < a->distinct; i++) {
       if (!strcmp(a->words[i].name, word)) {
```

```
            if (a->words[i].count == 2147483647) return 0;
            a->words[i].count++; a->total++; return 1;
        }
    }
    if (a->distinct == a->room) {
        size_t next = a->room ? a->room * 2 : 16;
        if (next < a->room || next > SIZE_MAX / sizeof(WordStat)) return 0;
        WordStat *p = realloc(a->words, next * sizeof *p);
        if (!p) { puts("Word memory allocation failed."); return 0; }
        a->words = p; a->room = next;
    }
    strcpy(a->words[a->distinct].name, word);
    a->words[a->distinct].count = 1;
    a->distinct++; a->total++; return 1;
}
int letter(unsigned char c) { return (c >= 'a' && c <= 'z') || (c >= 'A' && c
<= 'Z'); }
/* Walk through the text using pointer arithmetic. */
int analyze_text(Analyzer *a) {
    if (!a->text) return 1;
    const unsigned char *p = (const unsigned char *)a->text;
    while (*p) {
        while (*p && !letter(*p)) p++;
        if (!*p) break;
        char word[40]; size_t n = 0;
        while (letter(*p)) {
            if (n < 39) word[n] = (char)tolower(*p);
            n++; p++;
        }
        if (n > 39) { a->rejected++; continue; }
        word[n] = '\0';
        if (!count_word(a, word)) return 0;
    }
    return 1;
}
void replace_text(Analyzer *current, Analyzer *next) {
    if (!analyze_text(next)) {
        puts("Analysis failed; previous text kept.");
        free(next->text); free(next->words); return;
    }
    free(current->text); free(current->words);
    *current = *next;
```

```
   printf("Text ready: %zu words, %zu distinct, %zu overlong rejected.\n",
          current->total, current->distinct, current->rejected);
}
void load_example(Analyzer *a) {
   Analyzer next = {0};
   if (append(&next, sample)) replace_text(a, &next);
   else free(next.text);
   puts("Source: The Wonderful Wizard of Oz, Chapter 1, L. Frank Baum
(1900)");
}
void paste_text(Analyzer *a) {
   Analyzer next = {0}; char line[256];
   puts("Paste text; finish with END on its own line:");
   while (fgets(line, sizeof line, stdin)) {
       if (!strcmp(line, "END\n") || !strcmp(line, "END")) break;
       if (!append(&next, line)) { free(next.text); return; }
   }
   replace_text(a, &next);
}
void word_count(Analyzer *a) {
   printf("Words: %zu | Distinct: %zu | Rejected: %zu\n",
          a->total, a->distinct, a->rejected);
}
WordStat *longest(Analyzer *a) {
   WordStat *best = NULL;
   for (WordStat *p = a->words; p && p < a->words + a->distinct; p++)
       if (!best || strlen(p->name) > strlen(best->name)) best = p;
   return best;
}
WordStat *frequent(Analyzer *a) {
   WordStat *best = NULL;
   for (WordStat *p = a->words; p && p < a->words + a->distinct; p++)
       if (!best || p->count > best->count) best = p;
   return best;
}
void show_longest(Analyzer *a) {
   WordStat *p = longest(a);
   if (p) printf("Longest: %s (%zu letters)\n", p->name, strlen(p->name));
   else puts("No words.");
}
void show_frequent(Analyzer *a) {
   WordStat *p = frequent(a);
```

```
    if (p) printf("Most frequent: %s (%d times)\n", p->name, p->count);
    else puts("No words.");
}
void statistics(Analyzer *a) {
    if (!a->distinct) { puts("No words."); return; }
    for (size_t i = 0; i < a->distinct; i++)
        printf("%-39s %d\n", a->words[i].name, a->words[i].count);
}
int alphabetic(const void *x, const void *y) {
    return strcmp(((const WordStat *)x)->name, ((const WordStat *)y)->name);
}
int by_frequency(const void *x, const void *y) {
    const WordStat *a = x, *b = y;
    return a->count == b->count ? strcmp(a->name, b->name) : (a->count <
b->count) - (a->count > b->count);
}
void sort_words(Analyzer *a) {
    if (!a->distinct) { puts("No words."); return; }
    char s[10];
    if (!input("Sort 1=alphabet, 2=frequency: ", s, sizeof s)) return;
    if (strcmp(s, "1") && strcmp(s, "2")) { puts("Invalid choice."); return; }
    qsort(a->words, a->distinct, sizeof(WordStat), !strcmp(s, "1") ? alphabetic
: by_frequency);
    statistics(a);
}
void search_word(Analyzer *a) {
    char s[80];
    if (!input("Search word: ", s, sizeof s)) return;
    size_t n = strlen(s);
    if (!n || n > 39) { puts("Use 1-39 letters."); return; }
    for (char *p = s; *p; p++) {
        if (!letter((unsigned char)*p)) { puts("Letters only."); return; }
        *p = (char)tolower((unsigned char)*p);
    }
    for (size_t i = 0; i < a->distinct; i++)
        if (!strcmp(a->words[i].name, s)) {
            printf("%s: %d occurrence(s)\n", s, a->words[i].count); return;
        }
    printf("%s: 0 occurrences\n", s);
}
double diversity(Analyzer *a) { return a->total ? 100.0 * a->distinct /
a->total : 0; }
```

```c
void custom(Analyzer *a) {
   printf("Lexical Diversity = distinct / total x 100 = %.2f%%\n",
diversity(a));
   puts("Higher values mean a larger share of different words.");
}
void addresses(Analyzer *a) {
   printf("Text buffer (dynamic text): %p\n", (void *)a->text);
   printf("Word array (dynamic statistics): %p\n", (void *)a->words);
   printf("First word entry: %p\n", a->distinct ? (void *)&a->words[0] :
NULL);
}
void summary(Analyzer *a) {
   size_t letters = 0;
   for (size_t i = 0; i < a->distinct; i++)
       letters += strlen(a->words[i].name) * (size_t)a->words[i].count;
   printf("\nSUMMARY | Total: %zu | Distinct: %zu\n", a->total, a->distinct);
   printf("Average word length: %.2f\n", a->total ? (double)letters / a->total
: 0);
   show_longest(a); show_frequent(a); custom(a);
   printf("Rejected overlong tokens: %zu\n", a->rejected);
}
int main(void) {
   Analyzer a = {0};
   void (*operations[])(Analyzer *) = {
       word_count, show_longest, show_frequent, statistics, sort_words,
       search_word, summary, addresses, custom
   };
   printf("Student ID: %s\nProject: Dynamic Text Mining and Linguistic Pattern
Analyzer\n", STUDENT_ID);
   puts("Custom analysis: Lexical Diversity Score");
   load_example(&a);
   for (;;) {
       int choice;
       puts("\n1 Count  2 Longest  3 Frequent  4 Statistics  5 Sort  6
Search");
       puts("7 Summary  8 Addresses  9 Lexical diversity  10 Paste text  11
Reload sample  0 Exit");
       if (!get_choice(&choice)) { if (feof(stdin)) break; continue; }
       if (choice == 0) break;
       if (choice >= 1 && choice <= 9) operations[choice - 1](&a);
       else if (choice == 10) paste_text(&a);
       else if (choice == 11) load_example(&a);
```

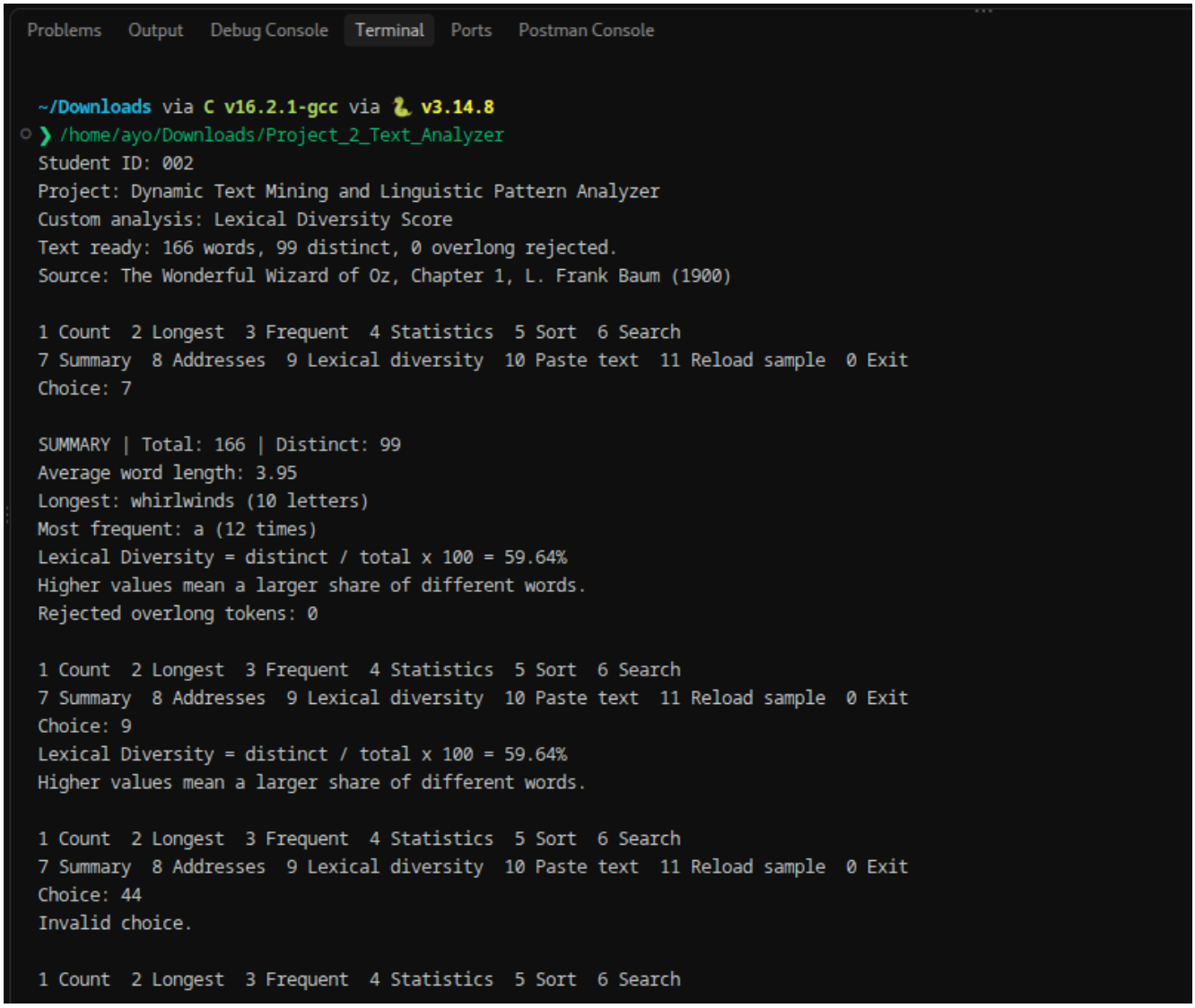
```
    }
    free(a.text); free(a.words);
    return 0;
}
```

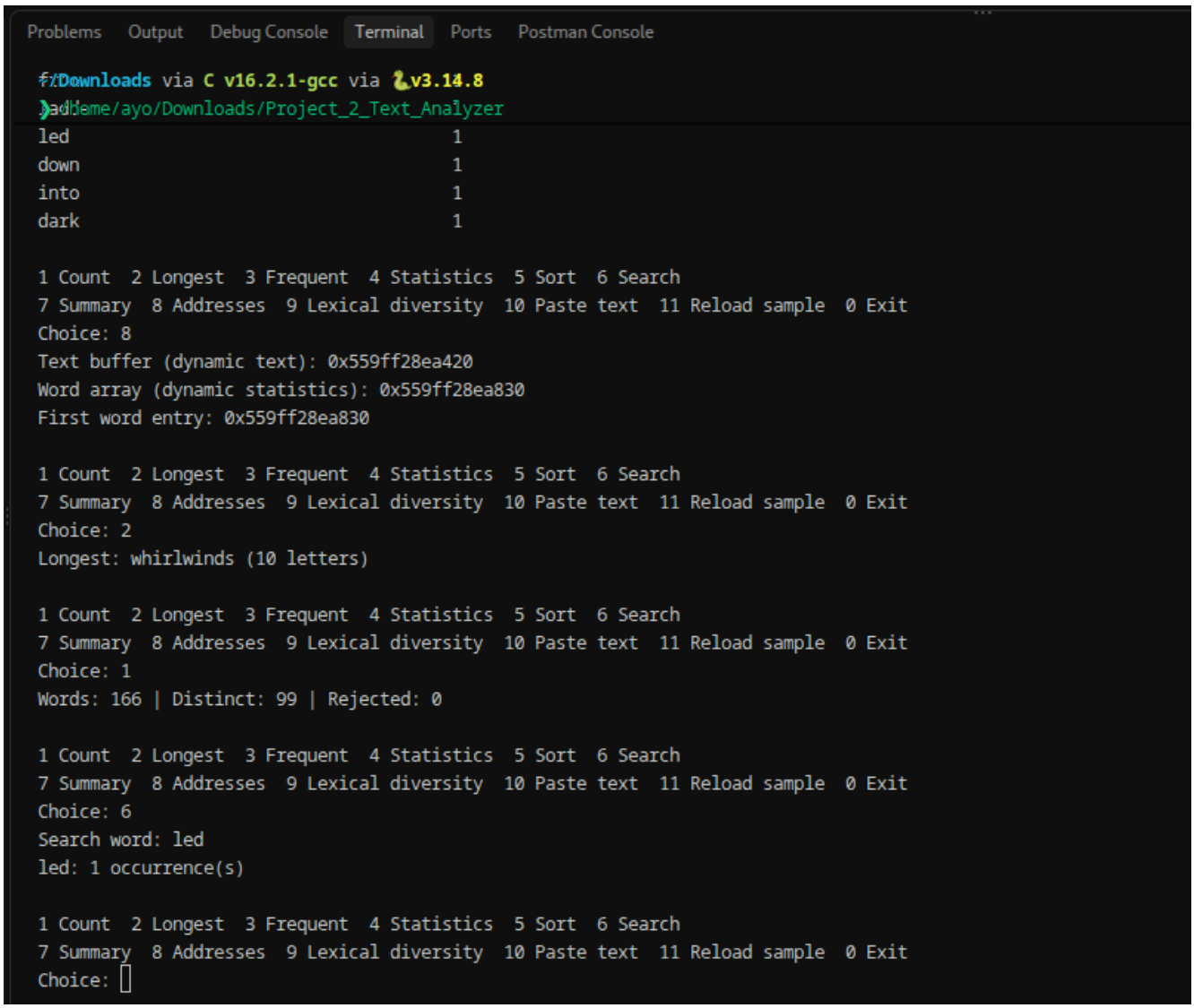
Selected Text and Source

Source: The Wonderful Wizard of Oz, Chapter 1, by L. Frank Baum (1900). The program includes a sample passage from this public-domain book and permits pasting replacement text. The sample contains 166 parsed words under the word-boundary rules below.

> Dorothy lived in the midst of the great Kansas prairies, with Uncle Henry, who was a farmer, and Aunt Em, who was the farmer's wife. Their house was small, for the lumber to build it had to be carried by wagon many miles. There were four walls, a floor and a roof, which made one room; and this room contained a rusty looking cookstove, a cupboard for the dishes, a table, three or four chairs, and the beds. Uncle Henry and Aunt Em had a big bed in one corner, and Dorothy a little bed in another corner. There was no garret at all, and no cellar-except a small hole dug in the ground, called a cyclone cellar, where the family could go in case one of those great whirlwinds arose, mighty enough to crush any building in its path. It was reached by a trap door in the middle of the floor, from which a ladder led down into the small, dark hole.

Sample Output from one test run





Short Technical Explanation

- **Dynamic storage:** A growing character buffer stores the text, and a separate dynamically allocated wordStat array stores each distinct word and frequency.
- **Pointer-based parsing:** The program scans text using a pointer and converts ASCII letters to lowercase. Punctuation, digits, apostrophes and hyphens mark word boundaries.
- **String handling:** strcmp() identifies duplicates, strlen() measures word length and strcpy() copies words after the parser confirms they fit the 39-character limit. Longer tokens are rejected and counted.
- **Function pointers:** A menu dispatches counting, sorting, searching, summary, address inspection and custom-analysis functions through an operation-pointer array.
- **Custom analysis - Lexical Diversity Score:** Lexical diversity = (number of distinct words / total number of words) x 100. The default example gives 99 / 166 x 100 = 59.64%. A larger percentage indicates a greater proportion of unique vocabulary.

- **Memory inspection and summary:** The program prints the text-buffer address, statistics-array address and an individual entry address using %p. The summary includes total words, distinct words, average word length, longest word and most frequent word.

Project 3: Dynamic IoT Event Monitoring and Callback Engine

Complete C Source Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <math.h>
#include <ctype.h>
#include <errno.h>
#include <limits.h>

#define STUDENT_ID "001"
enum { TEMP = 1, PRESSURE = 2, BATTERY = 3 };
enum { NORMAL = 0, ALERT = 1 };

typedef union {
   float temperature;
   int pressure;
   float voltage;
} Reading;

typedef struct Device {
   int id, type, status;
   char name[40];
   Reading reading;
   double previous, last_alert_value;
   int has_previous;
   size_t samples, alerts, drifts;
   char last_alert[170];
} Device;

typedef struct {
   Device *devices;
   size_t count, total, normal, abnormal, by_type[3], drift_alerts;
} Monitor;

int input(const char *message, char *s, size_t cap) {
   printf("%s", message);
   if (!fgets(s, (int)cap, stdin)) return 0;
```

```
    size_t n = strlen(s);
    if (n && s[n - 1] == '\n') s[n - 1] = '\0';
    else if (!feof(stdin)) {
        int c; while ((c = getchar()) != '\n' && c != EOF) {}
        puts("Input too long."); return 0;
    }
    return 1;
}
int number(const char *message, int *value) {
    char s[80], *end;
    if (!input(message, s, sizeof s)) return 0;
    errno = 0;
    long n = strtol(s, &end, 10);
    while (isspace((unsigned char)*end)) end++;
    if (errno || s == end || *end || n < INT_MIN || n > INT_MAX) {
        puts("Invalid whole number."); return 0;
    }
    *value = (int)n; return 1;
}
const char *type_name(int type) {
    return type == TEMP ? "Temperature" : type == PRESSURE ? "Pressure" :
"Battery";
}
const char *unit(int type) {
    return type == TEMP ? "C" : type == PRESSURE ? "kPa" : "V";
}
double value(Device *d) { /* Read only the active union member. */
    if (d->type == TEMP) return d->reading.temperature;
    if (d->type == PRESSURE) return d->reading.pressure;
    return d->reading.voltage;
}
void alert(Device *d, const char *callback, const char *range, const char
*reason) {
    d->status = ALERT;
    d->last_alert_value = value(d);
    snprintf(d->last_alert, sizeof d->last_alert, "Range: %s | Reason: %s |
Callback: %s",
             range, reason, callback);
    printf("ALERT | ID:%d | %s | %.2f %s | %s\n",
           d->id, d->name, value(d), unit(d->type), d->last_alert);
}
void temperature_monitor(Device *d) {
```

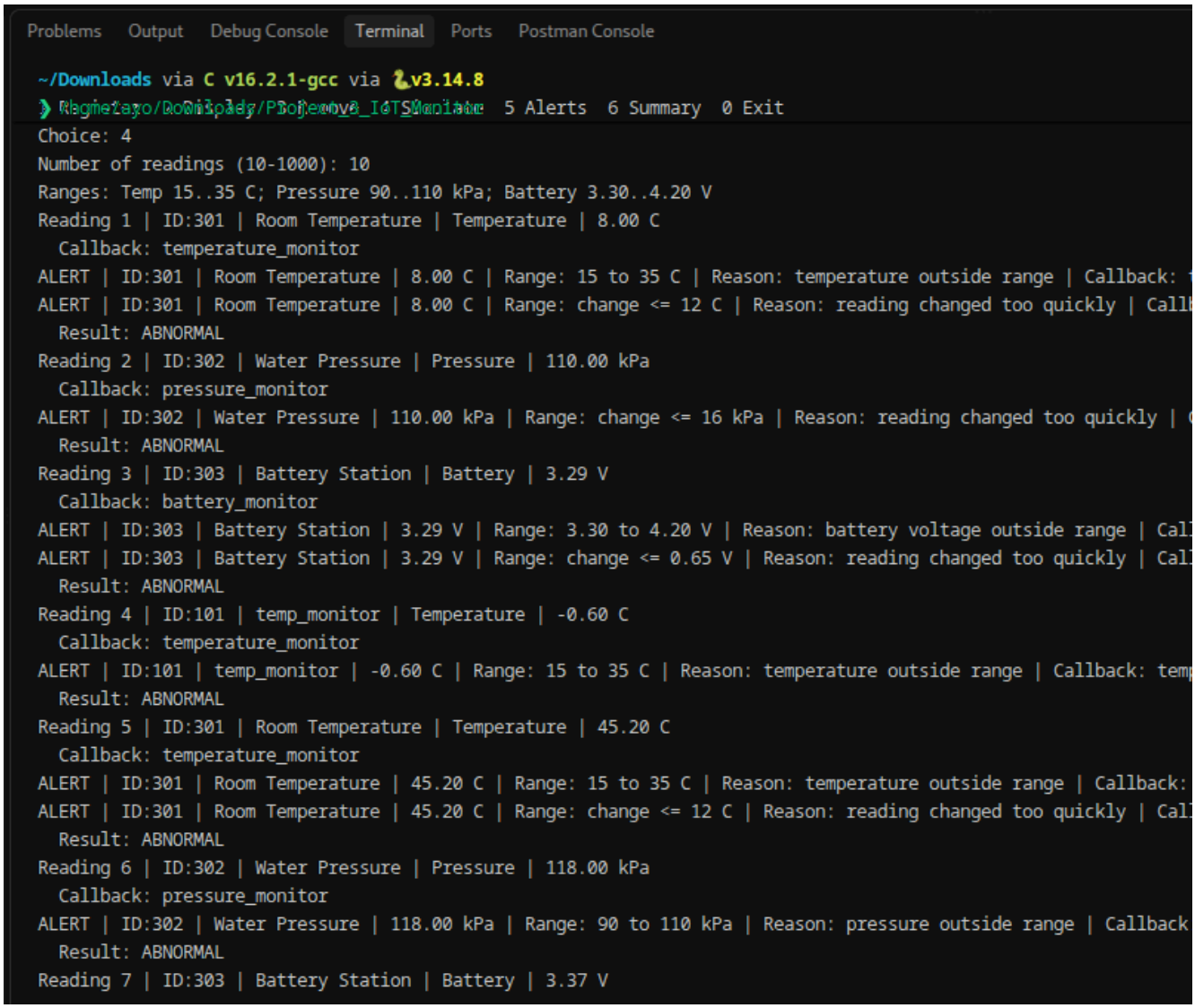
```c
    if (d->reading.temperature < 15 || d->reading.temperature > 35)
        alert(d, "temperature_monitor", "15 to 35 C", "temperature outside
range");
}
void pressure_monitor(Device *d) {
    if (d->reading.pressure < 90 || d->reading.pressure > 110)
        alert(d, "pressure_monitor", "90 to 110 kPa", "pressure outside
range");
}
void battery_monitor(Device *d) {
    if (d->reading.voltage < 3.3 || d->reading.voltage > 4.2)
        alert(d, "battery_monitor", "3.30 to 4.20 V", "battery voltage outside
range");
}
/* Extra callback: detect a large change since this device's previous reading.
   Thresholds: 12 C, 16 kPa, or 0.65 V. */
void drift_detector(Device *d) {
    double limit = d->type == TEMP ? 12 : d->type == PRESSURE ? 16 : 0.65;
    if (d->has_previous && fabs(value(d) - d->previous) > limit) {
        const char *range = d->type == TEMP ? "change <= 12 C" :
                            d->type == PRESSURE ? "change <= 16 kPa" : "change
<= 0.65 V";
        alert(d, "drift_detector", range, "reading changed too quickly");
        d->drifts++;
    }
}
void process_device(Device *d, void (*callback)(Device *)) { callback(d); }
int position(Monitor *m, int id) {
    for (size_t i = 0; i < m->count; i++) if (m->devices[i].id == id) return
(int)i;
    return -1;
}
void register_device(Monitor *m) {
    Device d = {0};
    if (!number("Device ID: ", &d.id)) return;
    if (d.id <= 0 || position(m, d.id) >= 0) { puts("ID must be positive and
unique."); return; }
    if (!input("Name: ", d.name, sizeof d.name) || !d.name[0]) return;
    if (!number("Type 1=temperature 2=pressure 3=battery: ", &d.type)) return;
    if (d.type < 1 || d.type > 3) { puts("Invalid device type."); return; }
    Device *p = realloc(m->devices, (m->count + 1) * sizeof *p);
    if (!p) { puts("Memory allocation failed."); return; }
```

```
    m->devices = p; m->devices[m->count++] = d;
    printf("Registered. Devices: %zu\n", m->count);
}
void display(Monitor *m) {
    if (!m->count) { puts("No devices."); return; }
    for (Device *d = m->devices; d < m->devices + m->count; d++)
        printf("%d | %-18.18s | %-11s | %s | Samples: %zu\n",
               d->id, d->name, type_name(d->type),
               !d->samples ? "NEW" : d->status ? "ALERT" : "NORMAL",
d->samples);
}
void remove_device(Monitor *m) {
    int id;
    if (!number("ID to remove: ", &id)) return;
    int i = position(m, id);
    if (i < 0) { puts("Device not found."); return; }
    for (size_t j = (size_t)i; j + 1 < m->count; j++) m->devices[j] =
m->devices[j + 1];
    m->count--;
    if (!m->count) { free(m->devices); m->devices = NULL; }
    else {
        Device *p = realloc(m->devices, m->count * sizeof *p);
        if (p) m->devices = p; /* failed shrinking retains valid records */
    }
    printf("Removed. Devices: %zu\n", m->count);
}
int random_between(int low, int high) { return low + rand() % (high - low +
1); }
void generate(Device *d) {
    /* Documented generation ranges:
       temperature -5..50 C, pressure 75..125 kPa, battery 2.80..4.60 V */
    if (d->type == TEMP) d->reading.temperature = random_between(-50, 500) /
10.0f;
    else if (d->type == PRESSURE) d->reading.pressure = random_between(75,
125);
    else d->reading.voltage = random_between(280, 460) / 100.0f;
}
void simulate(Monitor *m) {
    int n;
    if (!m->count) { puts("Register a device first."); return; }
    if (!number("Number of readings (10-1000): ", &n)) return;
    if (n < 10 || n > 1000) { puts("Choose 10 to 1000 readings."); return; }
```

```
   void (*callbacks[])(Device *) = {temperature_monitor, pressure_monitor,
battery_monitor};
   puts("Ranges: Temp 15..35 C; Pressure 90..110 kPa; Battery 3.30..4.20 V");
   for (int i = 0; i < n; i++) {
       Device *d = m->devices + i % m->count;
       generate(d);
       d->status = NORMAL;
       printf("Reading %d | ID:%d | %s | %s | %.2f %s\n",
              i + 1, d->id, d->name, type_name(d->type), value(d),
unit(d->type));
       const char *callback = d->type == TEMP ? "temperature_monitor" :
                              d->type == PRESSURE ? "pressure_monitor" :
"battery_monitor";
       printf("  Callback: %s\n", callback);
       process_device(d, callbacks[d->type - 1]);
       size_t old_drifts = d->drifts;
       process_device(d, drift_detector);
       m->drift_alerts += d->drifts - old_drifts;
       d->previous = value(d);
       d->has_previous = 1;
       d->samples++; m->total++;
       if (d->status == ALERT) {
           m->abnormal++; m->by_type[d->type - 1]++; d->alerts++;
           puts("  Result: ABNORMAL");
       } else { m->normal++; puts("  Result: NORMAL"); }
   }
}
void show_alerts(Monitor *m) {
   int found = 0;
   for (Device *d = m->devices; d < m->devices + m->count; d++) {
       if (!d->alerts) continue;
       found = 1;
       printf("ID:%d | %s | Alerts:%zu | Drift alerts:%zu\n",
              d->id, d->name, d->alerts, d->drifts);
       printf("  Most recent alert: %.2f %s | %s\n",
              d->last_alert_value, unit(d->type), d->last_alert);
   }
   if (!found) puts("No alerts. Full alert details are printed during
simulation.");
}
void summary(Monitor *m) {
```

```c
    printf("\nSUMMARY | Registered devices: %zu | Readings: %zu\n", m->count,
m->total);
    printf("Normal: %zu | Abnormal: %zu\n", m->normal, m->abnormal);
    printf("Alerts by type: Temp=%zu Pressure=%zu Battery=%zu\n",
           m->by_type[0], m->by_type[1], m->by_type[2]);
    printf("Custom callback - Sensor Drift Detector: %zu detections\n",
m->drift_alerts);
    puts("One reading counts once as abnormal even if both callbacks
trigger.");
}
int main(void) {
    Device initial[] = {
        {.id=301, .name="Room Temperature", .type=TEMP},
        {.id=302, .name="Water Pressure", .type=PRESSURE},
        {.id=303, .name="Battery Station", .type=BATTERY}
    };
    Monitor m = {0};
    m.count = sizeof initial / sizeof initial[0];
    m.devices = malloc(sizeof initial);
    if (!m.devices) { puts("Memory allocation failed."); return 1; }
    memcpy(m.devices, initial, sizeof initial);
    srand((unsigned)time(NULL));
    void (*operations[])(Monitor *) = {register_device, display, remove_device,
                                       simulate, show_alerts, summary};
    printf("Student ID: %s\nProject: Dynamic IoT Event Monitoring and Callback
Engine\n", STUDENT_ID);
    puts("Custom callback: Sensor Drift Detector");
    for (;;) {
        int choice;
        puts("\n1 Register  2 Display  3 Remove  4 Simulate  5 Alerts  6
Summary  0 Exit");
        if (!number("Choice: ", &choice)) { if (feof(stdin)) break; continue; }
        if (choice == 0) break;
        if (choice >= 1 && choice <= 6) operations[choice - 1](&m);
        else puts("Invalid choice.");
    }
    free(m.devices);
    return 0;
}
```

Sample Output from one test run



```
1 Register  2 Display  3 Remove  4 Simulate  5 Alerts  6 Summary  0 Exit
Choice: 6

SUMMARY | Registered devices: 4 | Readings: 44
Normal: 7 | Abnormal: 37
Alerts by type: Temp=21 Pressure=10 Battery=6
Custom callback - Sensor Drift Detector: 20 detections
One reading counts once as abnormal even if both callbacks trigger.

1 Register  2 Display  3 Remove  4 Simulate  5 Alerts  6 Summary  0 Exit
Choice: 2
301 | Room Temperature   | Temperature | NORMAL | Samples: 12
302 | Water Pressure     | Pressure    | ALERT | Samples: 12
303 | Battery Station    | Battery     | NORMAL | Samples: 10
101 | temp_monitor       | Temperature | ALERT | Samples: 10

1 Register  2 Display  3 Remove  4 Simulate  5 Alerts  6 Summary  0 Exit
Choice:
```

Short Technical Explanation

- Structures and unions: Each Device stores an ID, name, type, status, union reading and recent history. The union holds temperature, pressure or voltage according to the device type.
- **Dynamic device management:** malloc() initializes the device array and realloc() grows or shrinks it as devices are added or removed. Duplicate IDs are rejected and memory is released when the program ends.
- **Random simulation:** The program can generate 10 to 1000 readings. It uses rand() seeded by time(). Generation ranges are -5 to 50 C, 75 to 125 kPa and 2.80 to 4.60 V.
- **Callback processing:** Separate temperature, pressure and battery callbacks evaluate operating ranges of 15-35 C, 90-110 kPa and 3.30-4.20 V. A function pointer selects and invokes the correct monitor.
- **Custom callback - Sensor Drift Detector:** This callback compares a device reading with its previous reading. It flags absolute changes above 12 C for temperature, 16 kPa for pressure or 0.65 V for battery. It is invoked through a function pointer.
- **Alerts and summary:** Alerts show device information, readings, allowed limits, a reason and the callback. The summary counts devices, processed readings, normal/abnormal readings, alerts by type and custom drift detections.